



Dhaka Tesla Pool

Share a seat. Split the fare. Survive Dhaka traffic.

1. The Banani Rush-Hour Story

8:41 AM, Banani Road 11. **Jashim** is leaning against **Bullet**, his three-seat, battery-powered, entirely unaffiliated “Tesla.” **Nusrat**, already late, books a ride to Mohakhali. Two minutes later a total stranger named **Rafiq** books almost the same route to Gulshan 1. The app now has to figure out, in about a second, whether these two can share a seat, split the fare fairly, and survive a ten-minute ride without any of it getting weird. Then **Shirin** tries to grab the last seat thirty seconds later, and things get properly interesting. Jashim just wants to know who's actually riding and when he can go. Everyone else just wants to get where they're going, pay a fair price, and not accidentally make a new friend.

Steal this cast for your seed data and demo, or bring your own - just be consistent about it. Either way, spare the evaluator another `user1/driver1`; nobody's ever been charmed by a user named `user1`.

2. The Product Problem

Nusrat wants to get from Banani to Mohakhali. Rafiq wants to get from Banani to Gulshan 1. Jashim's Bullet has three seats. Passengers should be able to request a ride and, when it makes sense, share a Tesla with someone else. The driver needs to see who's assigned to the ride and what stage it's at. Each passenger needs to see their own fare and their own status, not anyone else's. And once a ride wraps up, the system should hold onto enough history to explain exactly what happened, in case anyone asks later.

3. Your Mission: Build the MVP

Build an MVP around three actors: **Passenger** (Nusrat, Rafiq, Shirin), **Driver/Tesla** (Jashim, Bullet), and **Ride/Pool**. You do not need to solve real routing - we're evaluating engineering judgment, not a Google Maps rebuild.

Passenger	Driver / Tesla	Pool / Ride Split
• Sign up/in • Request ride: pickup, destination, seats • See estimated fare • Track status: waiting → matched → in progress → completed/cancelled • View history; cancel while valid	• Sign in; go online/offline • Own a Tesla with fixed capacity • See relevant requests; accept a ride/pool • Mark arrival, start, complete trip • See passengers/seats and ride history	• Multiple requests may share one Tesla • Occupied seats never exceed capacity • Each passenger gets an individual fare • Clear lifecycle; obvious pool membership

Suggested lifecycle (improve it if you can explain why):

REQUESTED → MATCHED/ACCEPTED → DRIVER_ARRIVED → STARTED → COMPLETED (+ CANCELLED)

4. Keeping Geography Simple

Do not spend the challenge fighting map APIs. Keep it simple - a predefined list of Dhaka areas (Banani, Gulshan, Mohakhali, Dhanmondi, Mirpur, Uttara, Farmgate, Bashundhara, etc.), plain lat/long points, or a lightweight free map. Invent and document a matching rule (e.g. same pickup zone or compatible routes), and apply it consistently to Nusrat and Rafiq's overlapping-but-not-identical trip.

5. Fare Model - Keep It Understandable

Document a simple, testable model, e.g.:

```
passengerFare = baseFare + distanceCharge - poolDiscount
```

You can add traffic/weather/vehicle rules, but the evaluator must be able to test the calculation by hand using Nusrat and Rafiq's trip. Explain how you store money (integer paisa/poysha vs. decimal) and why. Payment: Cash or simulated **TeslaPay** wallet - no real gateway needed.

6. Technical Scope & Mandated Stack

Layer	Requirement	Notes
Frontend	React or Next.js	Next.js (App Router) recommended for routing/SSR; plain React + a router is fine.
Backend	Node.js	Express, NestJS, Fastify, or other - justify the pick (Section 7).
Database	Candidate's choice	Relational store (Postgres/MySQL/SQLite) recommended given pooling/capacity needs
Other tooling	Candidate's choice	ORM, validation, auth, test framework, hosting - all justified in README.

Backend - API/resource design, auth, validation, business-logic placement, error handling, ride state transitions, pool capacity enforcement, data consistency, code organization, logging, basic security. REST/GraphQL/other - explain your choice.

Frontend - correct flows/states, clear loading/error/empty states, reasonable component organization, API integration, usability. A simple, clean interface is enough.

Database - design the schema yourself (users, Teslas/vehicles+capacity, ride requests, pools, pool membership, status/history, fare, optional payment/rating/audit). Use proper relationships, constraints, indexes, and types; be ready to explain every table.

Docker - must run via `docker compose up`: app container(s), DB container, `.env.example`, migrations, seed data (use Jashim/Nusrat/Rafiq), health checks if you can.

Deployment - free/free-tier only, do not pay. If free backend hosting isn't available, document the constraint and give a reproducible Docker deployment instead. Public deployment preferred.

7. Technology Choice & Justification

For every non-mandated choice (DB, ORM, auth, styling, tests, hosting), your README needs: what you picked and the realistic alternatives; why it fits a ride-pooling MVP specifically; what would make you switch later. A trendy stack you can't defend earns nothing extra - and will cost you in the interview when we ask why.

8. AI Usage Policy - Yes, AI Is Allowed

Use ChatGPT, Claude, Copilot, Cursor, documentation, Stack Overflow, or other legitimate tools - do not

hide their use. AI is a normal engineering tool. **But if AI writes it, you still own it:** be ready to explain, debug, redesign, or modify any part live - what the code does, why the architecture and database look the way they do, how auth and pooling/capacity are enforced, how the app fails, and how you'd change it.

README AI Usage section: which tools you used, what for, one accepted suggestion, one rejected/changed suggestion and why. We do not score by "least AI used" - we score engineering understanding.

9. Architecture First

Before implementing everything, think the system through. Include an architecture diagram (Mermaid/Excalidraw/draw.io/image) showing at minimum `Browser` → `Next.js/React` → `Node.js API` → `Database`, plus an ERD. Your implementation should broadly match the documented architecture; update the docs if it changes. Do not introduce microservices, Kafka, Kubernetes, Redis, or queues just to look advanced - add complexity only when there is a reason.

10. Git Workflow - Part of the Assessment

The repository needs long-lived branches `master`, `pre-release`, and `release/<version>`, plus `feature/*` branches for actual feature work (e.g. `feature/passenger-auth`, `feature/tesla-pooling`, `feature/driver-flow`).

Flow: build one logical change on its feature branch with incremental commits → merge into `master` when it works → once MVP features are integrated, cut `pre-release` for integration fixes, docs, and deployment checks → cut `release/v1.0.0` from `pre-release` as the version shown in your video/deployment. We inspect the history: a perfect final repo with a meaningless history is weaker than a good repo showing a real engineering journey.

11. Commit Message Rules

Use `<type>(<scope>): <short description>` (feat/fix/refactor/test/docs/chore/build). One commit = one understandable logical change. Avoid "update / changes / fix / final / latest / working now / asdf" - and avoid fifty meaningless micro-commits only to satisfy the rule. We want a useful history, not Git theatre.

```
feat(auth): add passenger login endpoint
feat(pool): enforce Bullet's seat capacity
fix(pool): prevent overbooking available seats
build(docker): add compose setup for api and postgres
```

12. README, Testing, Concurrency & Bonus

README must cover, at minimum:

- Summary, problem statement, features implemented, screenshots/GIFs
- Architecture diagram and ERD/database diagram
- Tech stack, project structure, prerequisites
- Environment variables (`.env.example`, never real secrets)
- Local setup, Docker instructions, migration/seed instructions
- How to run frontend/backend and tests; demo credentials
- Deployment URL, API overview, key decisions/trade-offs, known limitations, next improvements
- AI Usage section and demo video link

Testing (meaningful, not coverage-chasing) should cover: Bullet's capacity can never be exceeded; invalid state transitions are rejected; Nusrat's and Rafiq's pooled fares calculate correctly; users can't modify another user's ride; cancellation rules hold; two concurrent requests can't corrupt pool capacity.

The concurrency problem: Bullet has 1 seat left. Nusrat and Shirin both try to claim it at nearly the same instant, and both initially see one seat available. Your MVP doesn't need a distributed solution, but your design should consider data consistency - document how you handle it now, and what you'd change at larger scale. Expect this in the interview.

Bonus - “If Oi Tesla Goes Viral”: without over-building the MVP, reason through scaling to 1M passengers and 100k drivers - load balancing, horizontal scaling, DB indexing/read replicas, caching, geospatial search, queues/events, real-time communication, rate limiting, idempotency, observability, DB contention, ride matching, retry/failure strategy, security, deployment strategy. A diagram is encouraged - reasoning matters more than box count.

13. Six-Minute Final Video

Record a **maximum 6-minute video** (Loom or similar free tool) and link it prominently in the README:
0:00-1:00 your understanding of the problem, users, and core idea in your own words (don't recite the PRD);
1:00-3:00 how you engineered it - architecture, backend, frontend, database design, the ride/pool lifecycle, one key decision, one trade-off, showing your architecture/ERD while explaining; **3:00-6:00** a product tour - passenger flow, driver flow, shared-Tesla/pooling, fare/status, one interesting edge case, deployment if available.

14. Submission Checklist

- Public/evaluator-accessible repo with a working MVP (frontend + backend + database)
- Docker setup, `.env.example`, no secrets committed
- Migrations and seed/demo data using the story cast
- Architecture diagram and ERD
- master / pre-release / release/v1.0.0 branches with a meaningful, incremental commit history
- Tests for important behavior, a self-explanatory README, deployment link if available
- Six-minute video link, AI Usage section, viral-scale bonus if attempted

15. What We Will Evaluate

We will not evaluate only whether the final screen “works.”

Product	Understood the problem, sensible assumptions
Process	Followed the instructions, git engineering, traceability
Backend / DB	API/state/validation design; modeling, constraints, integrity
Frontend	Correct flows/states, integration, maintainability
Docker / Deploy	Runs reliably elsewhere; shipped, not just coded
Testing / Docs	Tested what's risky; another engineer can operate it
Ownership	Can explain, defend, and change your own code

Following instructions is a major, explicit part of the score - 120 features with a broken process can score lower than a small, clean MVP that follows it properly.

16. What NOT to Do

Do not:

- Pay for infrastructure/services for this challenge
- Commit API keys, passwords, tokens, or `.env` secrets
- Submit a single giant “initial commit” containing the finished system
- Push all feature development directly to master
- Add technologies only to make the architecture diagram look impressive
- Polish animations while core data integrity is broken

- Hide AI usage, or include code you cannot explain
- Strip the story cast out of your seed data/tests/README in favor of generic placeholders

17. Assumptions Are Allowed

Some requirements are intentionally not fully specified - real engineering often starts with incomplete requirements. When something's unclear: make a reasonable assumption, document it, implement it consistently, and be ready to explain it. "Why did you assume that?" is not a trap - it's how we learn how you think.

18. Why the Details Matter

We wrote this brief with specific people and a specific vehicle for a reason: it's a lot harder to fake your way through a story than a spec sheet. A few things fall out of that naturally:

- Keep the cast consistent. If your seed data, tests, and demo still use Jashim, Bullet, Nusrat, and Rafiq (or your own cast, used the same way throughout), it shows you actually built the thing end to end rather than stitching together generic `user1/driver1` placeholders.
- Your git history tells us how you got there, not just where you ended up. A history that jumps from an empty repo straight to a finished app doesn't match the process this document lays out. • Be ready to talk through your own choices in the video and, if we get that far, in an interview - your schema, your state transitions, how you handled the concurrency problem in Section 14. That's hard to do convincingly for code you don't actually understand.
- Assumptions should be yours. Section 17 is there because the vaguer parts of this brief are exactly where a real engineer's thinking shows, and where a copy-pasted answer tends to fall apart. None of this is against using AI - Section 8 still stands. It's just that the bar here is understanding what you built, not just having something that runs.

19. Final Note From RoBenDevs

This challenge is designed to find future engineers, not candidates who have memorized the most frameworks. Use AI. Use documentation. Search things. Learn while building. But understand what you ship. We are interested in whether you can:

Understand → Design → Build → Commit → Test → Ship → Explain → Debug → Change

And remember: in Dhaka, your Tesla may have three wheels - but your engineering should still be production-minded.

Good luck, Chief Tesla Engineer.